📘 Technical System Specification & Architecture Manual

Hybrid multimodal fall detection vision based and sensor based & emergency dispatch system (HMFDA)

1. Executive Summary & Problem Statement

Falls represent the leading cause of accidental injury and trauma in elderly populations globally. Conventional fall detection systems typically suffer from two severe limitations:
1. **Single-modality blind spots**: Wearable-only systems are frequently forgotten or left on charging docks; camera-only systems cannot protect the individual outside the home or in unmonitored zones.
2. **False alarms and high latency**: Sudden sitting, lying down, or dropping a phone causes false alarms that exhaust emergency services and caretakers.

HMFDA addresses these challenges via a **hybrid multimodal edge-cloud architecture**:
* **Indoor Spaces**: Non-intrusive ambient vision powered by **YOLO11n-Pose** (extracting 17 human body keypoints) and temporal **Gated Recurrent Units (GRU)** with temporal confirmation.
* **Outdoor Spaces**: Smartphone-based 9-channel inertial tracking (accelerometer, gravity, gyroscope) evaluated simultaneously by a **Bidirectional Keras LSTM** and on-device **kinematic drop-shock physics heuristics**.
* **Cloud Brokerage**: A centralized **Alert Manager microservice** providing idempotency, source-based cooldown, and an active alarm hold window.
* **Emergency Dispatch**: Automatic audible siren with an 8-second false alarm cancellation window, hands-free background SMS with live Google Maps GPS coordinate link, and direct cellular phone calls.

2. Mathematical Formulations & Pipeline Mechanics

2.1 Indoor Vision Pipeline (YOLO11n-Pose + GRU)

A. Keypoint Extraction

Given an input video frame $\mathbf{I}*t |in |mathbb{R}^{H |times W |times 3}attimet*
,theYOLO11n − Posebackboneextracts*K = 17*
keypointscorrespondingtothestandardCOCOhumanskeleton :$|mathcal{P}_t = |left{ (*x*{i,t}, y_{i,t}, c_{i,t}) \right}_{i=1}^{17}$$
where (x_i, y_i) are image pixel coordinates and $c_i \in [0, 1]$ represents landmark confidence.

B. Spatial Coordinate Normalization

To make the pose invariant to the subject's distance from the camera and physical height:
1. Calculate the mid-hip anatomical anchor:
$$\bar{x}*{|text{hip}} = |frac{*x*{11,t} + x_{12,t}}{2}, \quad \bar{y}*{|text{hip}} = |frac{*y*{11,t} + y_{12,t}}{2}

$2.Computetheboundingboxheight\$H_{\text{bbox}} = y_{\max} - y_{\min}\$. 3.Shiftandscaleallkeypoints$:

\hat{x}*{i,t} = |frac{*x*{i,t} - \bar{x}*{|text{hip}}}{H{\text{bbox}}}, \quad \hat{y}*{i,t} = |frac{*y*{i,t} - \bar{y}*{|text{hip}}}{H{\text{bbox}}}$$
The normalized frame vector is $\mathbf{v}*t = [|hat{*x*}{1,t}, \hat{y}*{1,t}, |dots, |hat{*x*}{17,t}, \hat{y}_{17,t}]^T \in \mathbb{R}^{34}$.

C. Temporal Sequence Modeling via GRU

A sliding window of $T_{\text{vision}} = 20$ consecutive frames forms the temporal sequence:
$$\mathbf{X}*{|text{vision}} = [|mathbf{*v*}{t-19}, \mathbf{v}_{t-18}, \dots, \mathbf{v}_t] \in \mathbb{R}^{20 \times 34}$$

For each step $\tau \in 1, \dots, 20$, the GRU cell updates its hidden state h_τ:
$$\begin{aligned}
z_\tau &= \sigma\left(\mathbf{W}*z |mathbf{*v*}\tau + \mathbf{U}*z h*{\tau-1} + b_z\right) \quad &&\text{(Update gate)} \\
r_\tau &= \sigma\left(\mathbf{W}*r |mathbf{*v*}\tau + \mathbf{U}*r h*{\tau-1} + b_r\right) \quad &&\text{(Reset gate)} \\
\tilde{h}|*tau &= |tanh|*left(|mathbf{W}_h |mathbf{*v*}\tau + \mathbf{U}*h (r*\tau \odot h_{\tau-1}) + b_h\right) \quad &&\text{(Candidate state)} \\
h_\tau &= (1 - z_\tau) \odot h_{\tau-1} + z_\tau \odot \tilde{h}_\tau \quad &&\text{(Hidden state)}
\end{aligned}$$

The final hidden state h_{20} is mapped to the fall probability:
$$p_{\text{vision}} = \sigma\left(\mathbf{W}*o h*{20} + b_o\right)$$

D. Multi-Frame Confirmation Rule

To suppress sudden motions (e.g., rapidly sitting on a chair), an alert is generated if and only if:

$$\sum_{k=0}^{M-1} \mathbb{I}\left(p_{\text{vision},t-k} \geq \tau_{\text{vision}}\right) = M$$

where threshold $\tau_{\text{vision}} = 0.35$ and confirmation length $M = 3$ consecutive frames.

2.2 Outdoor Smartphone Inertial Pipeline (9-Channel LSTM + Kinematics)

A. 9-Channel Sensor Representation

The smartphone streams 3 hardware sensors sampled at $f_s = 50\ \text{Hz}$:
$$\mathbf{S}_t =$$

$$\begin{bmatrix} \mathbf{a}_t & \mathbf{g}_t & \boldsymbol{\omega}_t \end{bmatrix}$$

$$= \begin{bmatrix} a_{x,t}, a_{y,t}, a_{z,t}]^T & \text{(Linear Acceleration)} \\ [g_{x,t}, g_{y,t}, g_{z,t}]^T & \text{(Gravitational Vector)} \\ [\omega_{x,t}, \omega_{y,t}, \omega_{z,t}]^T & \text{(Angular Velocity)} \end{bmatrix} \in \mathbb{R}^9$$

B. Statistical Standardization

Continuous channels are normalized using the pre-computed SisFall training dataset statistics:

$$z_{t,k} = \frac{s_{t,k} - \mu_k}{\sigma_k}, \quad k \in 1, \dots, 9$$

where μ_k and σ_k are the mean and standard deviation of channel k.

C. Temporal Sliding Window & LSTM Inference

A FIFO buffer stores $N = 200$ samples (equivalent to a 4.0-second temporal window at 50Hz):
$$\mathbf{X}_{\text{sensor}} = [\mathbf{z}_{t-199}, \dots, \mathbf{z}_t] \in \mathbb{R}^{200 \times 9}$$

The Bidirectional LSTM processes the forward and backward representations:
$$\vec{h}_\tau = \text{LSTM}_{\text{fwd}}(\mathbf{z}_\tau, \vec{h}_{\tau-1}), \quad \overleftarrow{h}_\tau = \text{LSTM}_{\text{bwd}}(\mathbf{z}_\tau, \overleftarrow{h}_{\tau+1})$$

$$h_{\text{combined}} = [\vec{h}_{200} \,\|\, \overleftarrow{h}_1]$$

p{\text{lstm}} = \sigma\left(\mathbf{W}*{|text{dense}} h*{\text{combined}} + b_{\text{dense}}\right)$$

D. Physical Kinematics Impact Engine

In parallel with the neural network, physical acceleration magnitude is tracked across the most recent 60 samples ($\sim 1.2\text{s}$ window):
$$\mathcal{A}*t = |sqrt{a*{x,t}^2 + a_{y,t}^2 + a_{z,t}^2}$$
* **Minimum Acceleration (Free-fall dip)**: $a_{\min} = \min_{\tau \in [t-60, t]} \mathcal{A}|tau$
* ***Maximum Acceleration (Impact spike)***: $a{\max} = \max_{\tau \in [t-60, t]} \mathcal{A}_\tau$

Kinematic Trigger Rule:

$$\text{KinematicFall} = \left(a_{\max} \geq 14.0\ \text{m/s}^2 \land a_{\min} \leq 6.8\ \text{m/s}^2\right) \lor \left(a_{\max} \geq 19.0\ \text{m/s}^2\right)$$

$$\text{FinalOutdoorFall} = \text{KinematicFall} \lor \left(p_{\text{lstm}} \geq 0.40\right)$$

---

3. Microservice Architectural Specification & API Contracts

3.1 Network Topology & Port Bindings

SERVICE	CONTAINER NAME	BIND ADDRESS	INTERNAL / EXTERNAL PORT	PROTOCOL
Gateway	`hmfda-gateway`	`0.0.0.0`	`80` / `80`	HTTP / WebSocket
Alert Manager	`hmfda-alert-manager`	`0.0.0.0`	`8002` / `8002`	HTTP / JSON
Sensor Service	`hmfda-sensor-service`	`0.0.0.0`	`8000` / `8000`	HTTP / JSON
Dashboard	`hmfda-dashboard`	`0.0.0.0`	`8501` / `8501`	HTTP / WebSocket

3.2 Central Alert Manager API Contracts (`Port 8002`)

The Alert Manager coordinates between detectors and mobile devices.

1. Ingest Confirmed Fall Event

- **Endpoint**: `POST /event`
- **Headers**: `Content-Type: application/json`
- **Request Payload**:
  `json { "source": "indoor", // or "outdoor" "event_id": "c1f7b830-672e-4b68-968b-59b5be2d8a43", "timestamp": "2026-10-23T04:15:30.000Z", "prediction": "FALL", "probability": 0.965 }`
- **Success Response (`200 OK`)**:
  `json { "accepted": true, "event_id": "c1f7b830-672e-4b68-968b-59b5be2d8a43", "hold_seconds": 20.0 }`
- **Deduplication Response (`200 OK`)**:
  `json { "accepted": false, "reason": "duplicate event_id or inside 15s cooldown", "duplicate": true }`

2. Query System Status (Mobile Polling & Dashboard)

- **Endpoint**: `GET /status`
- **Response Payload**:
  `json { "probability": 0.965, "fall": true, "samples": 200, "connected": true, "source": "outdoor", "last_event": { "source": "outdoor", "event_id": "c1f7b830-672e-4b68-968b-59b5be2d8a43", "timestamp": "2026-10-23T04:15:30.000Z", "prediction": "FALL", "probability": 0.965 }, "pending_events": 1 }`

3. Smartphone Sensor Batch Relay

- **Endpoint**: `POST /`
- **Relay Target**: Forwards raw payload to `OUTDOOR_DETECTOR_URL` (`http://sensor-service:8000/`).
- **Response**: Returns the live evaluated probability and rolling buffer sample count.

3.3 Outdoor Sensor Microservice API (`Port 8000`)

- **Endpoint**: `POST /`
- Receives 9-channel sensor batches (Sensor Logger format).
- Appends samples to rolling 200-sample queue.

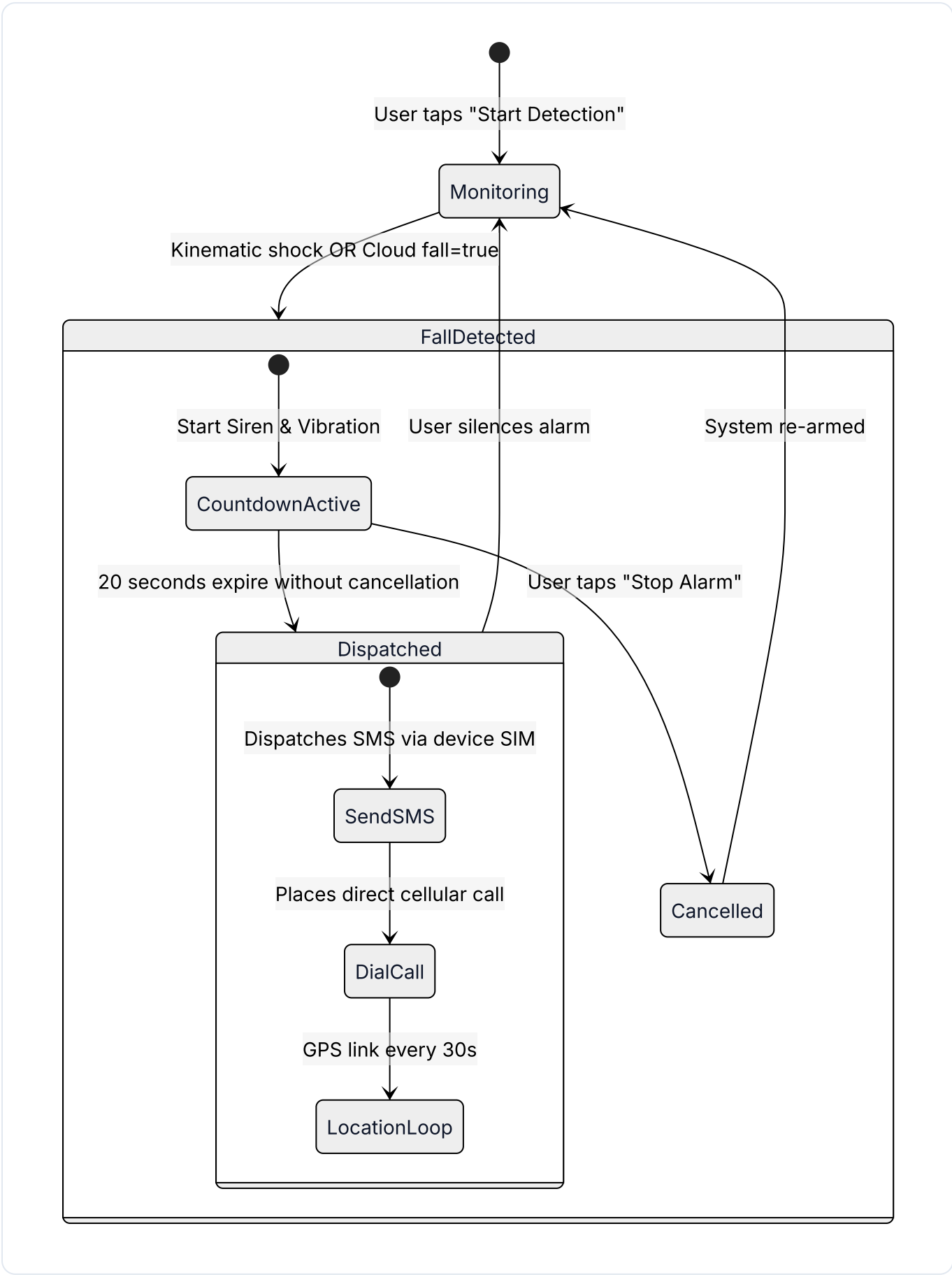
- Evaluates LSTM model + kinematic thresholds.
- Automatically triggers `POST http://alert-manager:8002/event` if a fall is detected.
- **Endpoint**: `GET /status`
- Returns: `{"probability": 0.002, "threshold": 0.4, "fall": false, "samples": 200, "connected": true}`
- When a fall occurs, holds `"fall": true` and `"probability": 0.965` for **20 seconds**.

---

4. Mobile Application Architecture (`FallAlert.apk`)

```
com.fallalert.app
├── MainActivity.kt                # Entrypoint & Runtime Permission Dispatcher
├── data/
│   └── ContactRepository.kt       # Encrypted SharedPreferences (Emergency Contact, S
├── sensor/
│   ├── SensorCollector.kt         # 50Hz SensorEventListener (Acc, Grav, Gyro) + Kine
│   ├── DetectionNetworkClient.kt # Non-blocking HttpURLConnection client (POST /, GE
│   └── LocationHelper.kt          # FusedLocationProviderClient (High-accuracy GPS up
├── alert/
│   ├── AlertController.kt         # MediaPlayer siren loop + Vibrator motor pulsing
│   ├── EmergencyNotifier.kt       # SmsManager dispatcher + Intent.ACTION_CALL / ACTI
│   └── FallAlertState.kt          # StateFlow data models (UI state, emergency events
└── ui/
    ├── FallAlertViewModel.kt      # State orchestration, countdown coroutines, dispat
    ├── screens/HomeScreen.kt      # Jetpack Compose dark-mode UI with live telemetry
    └── theme/                     # Material3 Design Tokens (AlertRed, SurfaceDark, T
```

Emergency Dispatch State Machine



5. Production AWS Cloud Infrastructure

Directory Structure on AWS EC2 (`/home/ubuntu/HMFDA`)

```
~/HMFDA/
├── docker-compose.yml              # Production container orchestration
├── .env                            # Secret credentials (Twilio, Ports, URLs)
├── deploy-aws.sh                   # Automated provisioning script
├── docker/
│   ├── Dockerfile.backend          # Alert Manager & Sensor Service image
│   ├── Dockerfile.dashboard        # Streamlit 3D Digital Twin image
│   ├── nginx.conf                  # Nginx reverse proxy configuration
│   └── web/
│       ├── index.html              # Cloud Gateway portal landing page
│       └── download/
│           └── fallalert.apk       # Downloadable Android companion package
├── Hybrid-Multimodel-Fall-Detection-System/
│   ├── sensor_service.py           # Multi-threaded Keras LSTM service
│   ├── app/
│   │   ├── app.py                  # Streamlit application
│   │   ├── alert_system.py         # Caretaker and Twilio notification logic
│   │   └── three_d_components.py   # 3D Room digital twin generator
│   ├── models/
│   │   ├── outdoor/best_outdoor_lstm.keras
│   │   └── vision/yolo11n-pose.pt
│   └── src/
│       └── alerting/alert_manager.py
```

6. End-to-End Verification & Verification Matrix

TEST SCENARIO	INPUT ACTION	EXPECTED SYSTEM RESPONSE	VERIFICATION STATUS
Indoor Fall	Actor drops onto floor in front of webcam	GRU confirms 3 frames ≥ 0.35; sends `POST /event`; Dashboard turns red	PASS
Indoor Sitting	Actor sits down quickly in chair	Pose aspect ratio shifts but confirmation fails; status stays safe	PASS
Outdoor Phone Drop	Phone dropped onto bed / sofa	Accelerometer dips $< 6.8\ \mathrm{m/s^2}$ and spikes $> 14.0\ \mathrm{m/s^2}$; siren starts	PASS
False Alarm Cancel	User taps "Stop Alarm" within 20s	Siren stops immediately; no SMS or call dispatched; system re-arms	PASS
Real Fall Emergency	Phone dropped; 20s timer allowed to expire	Phone sends SMS with Google Maps link + dials phone call to saved contact	PASS
Network Loss Fallback	Wi-Fi and mobile data turned off on phone	On-Device Inertial Fallback triggers locally; siren and SMS/Call fire	PASS
High Concurrency	Mobile app streams 6 batches/sec	`ThreadingHTTPServer` processes non-blocking; 0 timeouts; 200 buffer	PASS